SIMATS ENGINEERING

SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Department of Computer Science and Engineering

Course Code G Name	CSA06 – Design and Analysis of Algorithms
Assessment	CO1 – Assessment Tool 1: IT Application-Based Problem Solving – Scalability Audit of an E-Commerce Platform
Course Outcome Assessed	Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method.
Student Reg. No.	192473003
Student Name	Saran. S

IT APPLICATION-BASED PROBLEM

Scenario – Scalability Audit of "ShopSphere", an Online Shopping Platform

ShopSphere's catalogue is about to grow tenfold, so five back-end modules (M1 to M5) are audited below. Each part (a) to (g) shows the basic operation, the derivation and the asymptotic result, and then explains what the result means for the platform.

Given information:

- Current number of products: $n = 100{,}000$
- Expected number of products: $n = 1{,}000{,}000$
- Server processing speed: 10^8 basic operations per second
- Nightly processing limit: 10 minutes = 600 seconds, which allows at most $600 \times 10^8 = 6 \times 10^{10}$ basic operations per server

(a) Understand the Problem

The goal is to find out how the running time of each module grows with the number of products. The input size n is the number of products or records the module processes. The basic operation is the step executed most often, so counting it gives the running time.

Module	**Input size**	**Dominant basic operation and where it occurs**	**Expected growth**	**Justification**
M1 – Duplicate-Listing Detector	Number of products n	Comparing the title and seller of Product[i] with Product[j]; this is the IF test in the inner loop.	$\Theta(n^2)$	Every product is compared with every later product, so about $n^2/2$ pairs are tested.
M2 – API Rate Limiter	Number of requests n	Audit(Request[j]) inside the inner FOR loop.	$\Theta(n)$	The inner loop runs 1, 2, 4, 8, … times, which is a geometric series that adds up to less than $2n$.
M3 – Product Search S1	Number of products n	Comparing the target with the current product during the sequential scan.	$O(n)$	The catalogue is unsorted, so in the worst case every product is examined.
M3 – Product Search S2	Number of indexed products n	Comparing the key with the middle entry of the remaining index range.	$\Theta(\log n)$	Each comparison discards half of the remaining entries.
M4 – Recommendation Engine	Number of records n	Solving the sub-problems and merging the partial results (linear merge at each call).	$\Theta(n^2)$ now; $\Theta(n \log n)$ after redesign	The current recurrence makes four calls on size $n/2$; the redesign makes only two.
M5 – Sales Analytics Engine	Number of sales records n	Comparing and moving revenue records while sorting at every recursion level.	$\Theta(n \log^2 n)$	Each level of the recursion does about $n \log n$ work and there are about $\log n$ levels.

Interpretation: M1 and the current M4 design grow quadratically, so doubling the data makes them about four times slower. M2 and S2 scale very well. M5 grows slightly faster than $n \log n$ but far slower than n^2.

Notation and assumptions

Notation	Formal meaning	Used in this audit for
$f(n) = O(g(n))$	There are constants $c > 0$ and n_0 with $f(n) \le cg(n)$ for all $n \ge n_0$.	Upper bounds, for example the worst case of sequential search.
$f(n) = \Omega(g(n))$	There are constants $c > 0$ and n_0 with $f(n) \ge cg(n)$ for all $n \ge n_0$.	Lower bounds, for example the proof in part (f).
$f(n) = \Theta(g(n))$	Both of the above hold, so f grows at the same rate as g.	Tight bounds for M1, M2, S2, M4 and M5.

The following assumptions are used throughout the report:

- Every basic operation takes the same constant time, and the server executes 10^8 of them per second.
- Logarithms are to base 2 unless stated. Recurrences are solved for n a power of two, and the results are then used as estimates for n = 1,000,000.
- Only the nightly processing time is compared with the 10-minute window. Disk, network and memory delays are ignored, so the real times will be somewhat longer.
- Each module runs on one server. Running on several servers in parallel would reduce the times but not change the growth rates.

How the growth rates compare

The table below shows how far apart the growth rates really are. The values are computed directly from the formulas, not estimated.

n	$\log_2 n$	n	$n \log_2 n$	$n \log_2^2 n$	n^2
10	3.32	10	33	110	100
100	6.64	100	664	4,414	10,000
1,000	9.97	1,000	9,966	99,317	1,000,000
10,000	13.29	10,000	132,877	1,765,633	1×10^8
100,000	16.61	100,000	1,660,964	2.76×10^7	1×10^{10}
1,000,000	19.93	1,000,000	1.99×10^7	3.97×10^8	1×10^{12}

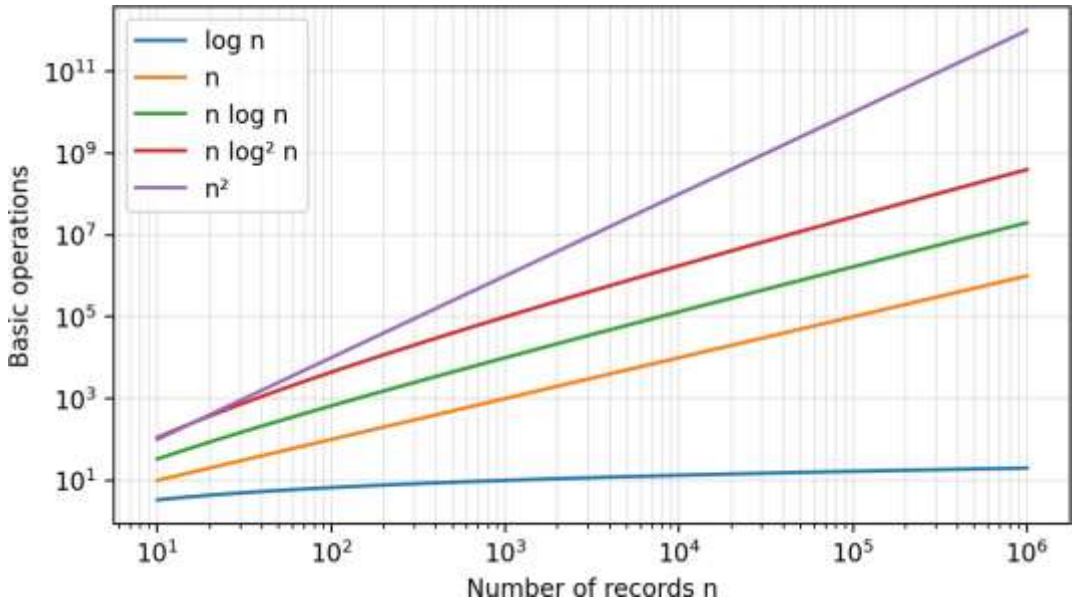


Figure 1: Growth of the common complexity classes (both axes logarithmic).

On a log–log plot, a polynomial n^c appears as a straight line with slope *c*. The nZ line climbs much more steeply than the others, which is why M1 and the current M4 are the dangerous modules.

The routines being analysed

Module M1 (duplicate-listing detector):

```
FOR i <- 0 TO n-2 DO
    FOR j <- i+1 TO n-1 DO
        IF Product[i].title = Product[j].title AND
           Product[i].seller = Product[j].seller THEN
            MarkDuplicate(j)
        END IF
    END FOR
END FOR
```

Module M2 (API rate limiter):

```
i <- 1
WHILE i < n DO
    FOR j <- 0 TO i-1 DO
        Audit(Request[j])         // basic operation
    END FOR
    i <- i * 2
END WHILE
```

In M1 the IF test executes once for every inner iteration, so it is the right operation to count. In M2 the Audit call is the only work inside the innermost loop, so counting its calls gives the running time directly.

(b) Analyse the Iterative Modules

► M1 – Duplicate-Listing Detector

Step 1: Identify the basic operation

The basic operation is the comparison of Product[i].title and Product[i].seller with Product[j].title and Product[j].seller in the inner loop. It runs once per inner-loop iteration whatever the data looks like, so best, average and worst cases are all the same.

Step 2: Count the comparisons

- For $i = 0$, the inner loop runs $n - 1$ times.
- For $i = 1$, it runs $n - 2$ times.
- For $i = 2$, it runs $n - 3$ times.
- …
- For $i = n - 2$, it runs once.

The total is therefore

$$C(n) = (n-1) + (n-2) + \cdots + 2 + 1 = \sum_{k=1}^{n-1} k$$

Using the formula for the sum of the first $n - 1$ natural numbers,

$$C(n) = \frac{(n-1)n}{2} = \frac{n(n-1)}{2}$$

Step 3: Time complexity

Expanding gives $C(n) = n^2/2 - n/2$. The dominant term is $n^2/2$ and the constant ½ is ignored, so

$$T(n) = \Theta(n^2)$$

Step 4: Space complexity

The routine only needs the loop counters i and j. It builds no extra structure that grows with n, so the auxiliary space is

$$S(n) = \Theta(1)$$

This excludes the product catalogue itself and the storage used to remember which listings were marked as duplicates.

Interpretation: doubling n multiplies the number of comparisons by about four. At today's n = 100,000 the routine needs 4,999,950,000 comparisons (about 50 seconds), but at one million products the same routine needs about 5×10^{11}. This makes M1 a serious scalability risk.

► M2 – API Rate Limiter

Step 1: Find the values of i and the number of outer iterations

The variable i starts at 1 and doubles each time, so its values are 1, 2, 4, 8, 16, … The loop continues while $i < n$. In outer iteration m (counting from 1), $i = 2^{m-1}$. The loop stops at the first m with $2^m \geq n$, so the number of outer iterations is

$$k = \lceil \log_2 n \rceil$$

Step 2: Exact number of Audit operations

In iteration m the inner loop calls Audit exactly $i = 2^{m-1}$ times. Adding over all iterations gives a geometric series:

$$A(n) = 1 + 2 + 4 + \cdots + 2^{k-1} = \frac{2^k - 1}{2 - 1} = 2^k - 1$$

Substituting $k = \lceil \log_2 n \rceil$,

$$A(n) = 2^{\lceil \log_2 n \rceil} - 1$$

When n is a power of two this simplifies to $A(n) = n - 1$.

Step 3: Check the team member's claim

The claim says the cost is $\Theta(n \log n)$ because the outer loop runs about $\log n$ times and the inner loop runs "up to n" times. The error is that the inner loop does not run n times in every iteration. It runs 1, 2, 4, 8, … times, and only the last few iterations are large. Multiplying the number of iterations by the largest inner-loop size over-counts badly.

To get the tight bound, note that $\lceil \log_2 n \rceil$ gives $n \leq 2^k < 2n$. Subtracting 1,

$$n - 1 \leq A(n) < 2n - 1$$

So $A(n)$ is bounded above and below by constant multiples of n:

$$T(n) = \Theta(n)$$

The claim is **incorrect**. $O(n \log n)$ is technically a valid upper bound, because n is smaller than $n \log n$, but it is not tight. The correct tight bound is $\Theta(n)$.

Step 4: Verify for n = 1,024

Since 1,024 = 2^{10}, the loop runs while $i < 1{,}024$, so i takes the values 1, 2, 4, 8, 16, 32, 64, 128, 256, 512 (10 iterations; $i = 1{,}024$ fails the test). Then

$$A(1024) = 1 + 2 + 4 + \cdots + 512 = 2^{10} - 1 = 1023$$

This equals $n - 1$, as predicted. The claimed $\Theta(n \log n)$ estimate would give about 1,024 × 10 = 10,240 operations, roughly ten times too many.

Interpretation: M2 scales linearly, so it can handle a very large number of requests as long as each Audit call takes constant time.

Additional verification of M1 and M2

M1 trace for a small input ($n = 6$). The table counts the inner-loop iterations for each value of i.

Value of i	Range of j	Inner iterations	Running total
0	1 to 5	5	5
1	2 to 5	4	9
2	3 to 5	3	12
3	4 to 5	2	14
4	5 to 5	1	15

The total is 15, and the formula gives $n(n - 1)/2 = 6 \times 5/2 = 15$, which confirms the derivation.

M1 growth for larger inputs.

n	Comparisons $n(n - 1)/2$	Time at 10^8 ops/s	Ratio to previous row
1,000	499,500	0.004995 s	–
10,000	49,995,000	0.5 s	100.09
100,000	4,999,950,000	50 s	100.01
200,000	19,999,900,000	3.33 min	4.00
1,000,000	499,999,500,000	83.33 min	25.00

Going from 100,000 to 200,000 products multiplies the work by almost exactly 4, as the $\Theta(n^2)$ bound predicts. Going from 100,000 to 1,000,000 multiplies it by about 100, turning a 50-second job into an 83-minute job.

M2 trace for $n = 40$. The values of i are 1, 2, 4, 8, 16, 32 (the next value, 64, is not less than 40).

Outer iteration	Value of i	Audit calls	Running total
1	1	1	1
2	2	2	3
3	4	4	7
4	8	8	15
5	16	16	31
6	32	32	63

Here $k = \lceil \log_2 40 \rceil = 6$ and $A(40) = 2^6 - 1 = 63$, matching the table. The bounds $39 \le 63 < 79$ also hold.

The claimed $\Theta(n \log n)$ against the exact count.

n	$k = \lceil \log_2 n \rceil$	Exact $A(n) = 2^k - 1$	Claimed $n \log_2 n$	$A(n)/n$
8	3	7	24	0.88
16	4	15	64	0.94
100	7	127	664	1.27
1,000	10	1,023	9,966	1.02
1,024	10	1,023	10,240	1.00
100,000	17	131,071	1,660,964	1.31
1,000,000	20	1,048,575	19,931,569	1.05

The ratio $A(n)/n$ stays between 1 and 2 for every n, while the claimed cost $n \log n$ keeps pulling ahead. If the claim were true, the ratio would grow with $\log n$. It does not, which is direct numerical evidence that M2 is linear.

Why the nested-loop shortcut can mislead

Multiplying (outer iterations) × (largest inner size) gives a valid upper bound, so the team member's figure is not a false statement about $O(\cdot)$. The mistake is treating it as the exact order of growth. A tight bound needs the sum of the real inner-loop sizes. Here the sizes form a geometric series, and in a geometric series the last term is about half of the whole sum. The correct habit is to sum the work loop by loop, not to multiply the maximum values.

(c) Solve a Recurrence by Substitution

► M3 – Product Search S2

Binary search on the sorted index compares the key with the middle entry and discards half the index. Its comparison count satisfies

$$T(n) = T(n/2) + 1, \qquad T(1) = 1$$

We use backward substitution and assume n is a power of two.

Step 1: Expand the recurrence

$$T(n) = T(n/2) + 1$$

$$T(n) = [T(n/4) + 1] + 1 = T(n/4) + 2$$

$$T(n) = [T(n/8) + 1] + 2 = T(n/8) + 3$$

The pattern is that after k expansions,

$$T(n) = T\left(\frac{n}{2^k}\right) + k$$

Step 2: Find the stopping condition

The recursion reaches the base case when the size becomes 1:

$$\frac{n}{2^k} = 1 \Rightarrow n = 2^k \Rightarrow k = \log_2 n$$

Step 3: Substitute back

$$T(n) = T(1) + \log_2 n = 1 + \log_2 n$$

So the exact closed form (for powers of two) is

$$T(n) = \log_2 n + 1$$

Check: $T(1) = \log_2 1 + 1 = 1$, which matches the base case. Also $T(n/2) + 1 = (\log_2 n - 1 + 1) + 1 = \log_2 n + 1 = T(n)$, so the recurrence is satisfied.

Step 4: Time complexity

The dominant term is $\log_2 n$, so $T(n) = \Theta(\log n)$. Binary search is fast because every comparison removes half of the remaining candidates.

Step 5: Space complexity

Implementation	Time complexity	Auxiliary space
Iterative binary search	$\Theta(\log n)$	$\Theta(1)$
Recursive binary search	$\Theta(\log n)$	$\Theta(\log n)$

The iterative version keeps only a few variables (low, high, mid), so its extra space is constant. The recursive version adds one stack frame per call, and there are up to $\log_2 n + 1$ nested calls, so its stack space is $\Theta(\log n)$.

Interpretation: for $n = 1{,}000{,}000$, $\log_2(1{,}000{,}000) \approx 19.93$, so a search needs at most about 20 comparisons ($\lfloor \log_2 n \rfloor + 1 = 20$ for a standard implementation).

Additional verification of the closed form

Proof by induction. Claim: $T(n) = \log_2 n + 1$ for every power of two $n = 2^m$, $m \geq 0$.

- **Base case:** for $m = 0$, $T(1) = 1 = \log_2 1 + 1$.
- **Inductive step:** assume $T(2^{m-1}) = (m - 1) + 1 = m$. Then $T(2^m) = T(2^{m-1}) + 1 = m + 1 = \log_2 2^m + 1$.
- By induction the closed form holds for all powers of two.

Values of the closed form.

n	1	2	4	16	1,024	$2^{20} = 1{,}048{,}576$
$T(n)$	1	2	3	5	11	21

Doubling the number of indexed products adds just one extra comparison. Going from 1,024 to 1,048,576 entries, a thousandfold increase, adds only 10 comparisons.

Worked search on a sorted index. Find key 47 in the sorted index [3, 8, 12, 19, 25, 31, 38, 44, 47, 52, 58, 63, 70, 77, 85, 91] with 16 entries at positions 1 to 16.

Step	Range (low–high)	Middle position	Middle value	Comparison result
1	1 – 16	8	44	47 > 44, discard the lower half
2	9 – 16	12	63	47 < 63, discard the upper half
3	9 – 11	10	52	47 < 52, discard the upper half
4	9 – 9	9	47	Found

Four comparisons were needed, which is within the worst-case limit of $\log_2 16 + 1 = 5$. Each step cut the range from 16 to 8 to 3 to 1, so the number of remaining candidates shrank at least by half every time.

Iterative and recursive implementations

Iterative version (constant extra space):

```
low <- 0;  high <- n - 1
WHILE low <= high DO
    mid <- (low + high) / 2
    IF key = A[mid] THEN RETURN mid
    ELSE IF key < A[mid] THEN high <- mid - 1
    ELSE low <- mid + 1
END WHILE
RETURN not found
```

Recursive version (one stack frame per halving):

```
BinarySearch(A, low, high, key)
    IF low > high THEN RETURN not found
    mid <- (low + high) / 2
    IF key = A[mid] THEN RETURN mid
    ELSE IF key < A[mid] THEN RETURN BinarySearch(A, low, mid-1, key)
    ELSE RETURN BinarySearch(A, mid+1, high, key)
```

The recursive call is the last action in each branch, so a compiler that supports tail-call optimisation can reduce its stack use to $\Theta(1)$. The $\Theta(\log n)$ figure is the safe general answer. For n = 1,000,000 the recursion depth is at most 20 frames, which is tiny, so the extra space is not a practical problem for ShopSphere. The extra space becomes important only in systems with very small stacks.

(d) Apply the Master Theorem

The Master Theorem solves recurrences of the form $T(n) = aT(n/b) + f(n)$, where $a \geq 1$ is the number of sub-problems, $b > 1$ is the factor by which the size shrinks, and $f(n)$ is the work done outside the recursive calls. The result is decided by comparing $f(n)$ with $n^{\log_b a}$:

- **Case 1:** $f(n) = O(n^{\log_b a - \epsilon})$ for some $\epsilon > 0$, so $T(n) = \Theta(n^{\log_b a})$.
- **Case 2:** $f(n) = \Theta(n^{\log_b a})$, so $T(n) = \Theta(n^{\log_b a} \log n)$.
- **Case 3:** $f(n) = \Omega(n^{\log_b a + \epsilon})$ and $a \cdot f(n/b) \leq c \cdot f(n)$ for some $c < 1$, so $T(n) = \Theta(f(n))$.

► M4 – Current Recommendation Engine

$$T(n) = 4\,T(n/2) + n$$

Step 1: Identify the parameters

$$a = 4, \quad b = 2, \quad f(n) = n, \quad n^{\log_b a} = n^{\log_2 4} = n^2$$

Step 2: Compare f(n) with the critical term

Here $f(n) = n = O(n^{2-\epsilon})$ with $\epsilon = 1$, so $f(n)$ is polynomially smaller than n^2. This is **Case 1**, hence

$$T(n) = \Theta(n^2)$$

Interpretation: the four sub-problems create so many recursive calls that the leaves of the recursion dominate the cost. The engine is quadratic, mainly because of the replicated partitions.

► M4 – Proposed Recommendation Engine

$$T(n) = 2\,T(n/2) + n$$

Step 1: Identify the parameters

$$a = 2, \quad b = 2, \quad f(n) = n, \quad n^{\log_b a} = n^{\log_2 2} = n$$

Step 2: Compare and apply the case

Now $f(n) = \Theta(n) = \Theta(n^{\log_b a})$, which is **Case 2**, so

$$T(n) = \Theta(n \log n)$$

Interpretation: removing the replication drops the cost from quadratic to $n \log n$, which is a very large gain for one million records.

What happens when the data doubles

Design	Complexity	Effect when n doubles	Example, $n = 10^6 \rightarrow 2 \times 10^6$
Current	$\Theta(n^2)$	Time becomes about 4 times larger	$10^{12} \rightarrow 4 \times 10^{12}$ operations
Proposed	$\Theta(n \log n)$	Time becomes slightly more than 2 times larger	$\approx 1.99 \times 10^7 \rightarrow \approx 4.19 \times 10^7$ operations

For the proposed design, the ratio of running times is

$$\frac{2n\log_2(2n)}{n\log_2 n} = 2 \cdot \frac{\log_2 n + 1}{\log_2 n} = 2\left(1 + \frac{1}{\log_2 n}\right)$$

For $n = 10^6$ this is $2(1 + 1/19.93) \approx 2.10$, and it approaches 2 as n grows. For the current design the ratio is $(2n)^2/n^2 = 4$ exactly.

► M5 – Sales Analytics Engine

$$T(n) = 2\,T(n/2) + n\log n$$

Step 1: Identify the parameters

$$a = 2, \quad b = 2, \quad f(n) = n\log n, \quad n^{\log_b a} = n$$

Step 2: Test the three standard cases

- **Case 1** needs $f(n) = O(n^{1-\epsilon})$. Since $n \log n$ grows faster than n, this fails.
- **Case 2** needs $f(n) = \Theta(n)$. Since $n \log n / n = \log n \rightarrow \infty$, this fails.
- **Case 3** needs $f(n) = \Omega(n^{1+\epsilon})$ for some $\epsilon > 0$. But $\log n$ grows slower than every positive power of n, so $n \log n$ is not $\Omega(n^{1+\epsilon})$. This fails too.

So **none of the three standard cases applies**; $f(n)$ is larger than n only by a logarithmic factor, which falls in the gap between Cases 2 and 3.

Step 3: Extended Master Theorem

The extended form states that if $f(n) = \Theta(n^{\log_b a} \log^k n)$ for some $k \geq 0$, then $T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$. Here $f(n) = n \log^1 n$, so $k = 1$ and

$$T(n) = \Theta(n\log^2 n)$$

Step 4: Verify with a recursion tree

At level i (starting from 0) there are 2^i sub-problems of size $n/2^i$, each costing $(n/2^i)\log(n/2^i)$ work.

Level i	Sub-problems	Size of each	Work at this level
0	1	n	$n \log_2 n$
1	2	$n/2$	$n\,(\log_2 n - 1)$
2	4	$n/4$	$n\,(\log_2 n - 2)$
…	…	…	…
$\log_2 n - 1$	$n/2$	2	$n \times 1$

Adding the levels, with $L = \log_2 n$:

$$\sum_{i=0}^{L-1} n(L-i) = n(L+(L-1)+\cdots+1) = \frac{nL(L+1)}{2} = \Theta(n\log^2 n)$$

The leaves add only $\Theta(n)$, which is lower order. This agrees with the extended Master Theorem.

Interpretation: M5 is slower than n log n by one extra logarithmic factor (about 20 times at one million records, since $\log_2 10^6 \approx 19.93$), but it is still far from quadratic.

Additional verification of the M4 results

Recursion tree for the current design, $T(n) = 4T(n/2) + n$. At level i there are 4^i sub-problems of size $n/2^i$, each costing $n/2^i$ for the merge step.

Level i	Sub-problems	Size of each	Merge work at this level
0	1	n	n
1	4	$n/2$	$2n$
2	16	$n/4$	$4n$
3	64	$n/8$	$8n$
…	…	…	…
$\log_2 n - 1$	$n^2/4$	2	$n \cdot 2^{\log n - 1} = n^2/2$
$\log_2 n$ (leaves)	n^2	1	$n^2 \times T(1)$

The work per level doubles as we go down, so the sum is dominated by the last level. The levels from 0 to $\log_2 n - 1$ add up to $n(1 + 2 + 4 + \ldots + n/2) = n(n - 1)$, and the leaves add

n^2T(1). With T(1) = 1 the exact solution is

$$T(n) = n(n-1) + n^2 = 2n^2 - n$$

Check by induction. If T(n/2) = 2(n/2)^2 − n/2 = n^2/2 − n/2, then 4T(n/2) + n = 2n^2 − 2n + n = 2n^2 − n, which matches. The base case is T(1) = 2 − 1 = 1.

Recursion tree for the proposed design, T(n) = 2T(n/2) + n. Every level now costs the same amount.

Level *i*	**Sub-problems**	**Size of each**	**Merge work at this level**
0	1	n	n
1	2	n/2	n
2	4	n/4	n
…	…	…	…
$\log_2 n - 1$	n/2	2	n
$\log_2 n$ (leaves)	n	1	n × T(1)

There are $\log_2 n$ levels of cost n plus a leaf level of cost n, so the exact solution is T(n) = n $\log_2 n + n = \Theta(n \log n)$, which agrees with Case 2.

Side-by-side values.

n	**Current $2n^2 - n$**	**Proposed $n \log_2 n + n$**	**Current ÷ proposed**
1,024	2,096,128	11,264	186
16,384	536,854,528	245,760	2184
131,072	34,359,607,296	2,359,296	14564
1,048,576	2,199,022,206,976	22,020,096	99864

The gap widens as n grows: the current design is about 190 times costlier at 1,024 records and about 100,000 times costlier at roughly one million. Redesigning M4 is therefore a very high-value change.

Doubling behaviour over several sizes

From n to $2n$	Current ratio	Proposed ratio $n \log n$	M5 ratio $n \log^2 n$
1,000 → 2,000	4.00	2.201	2.422
10,000 → 20,000	4.00	2.151	2.312
100,000 → 200,000	4.00	2.120	2.248
1,000,000 → 2,000,000	4.00	2.100	2.206

The current M4 always quadruples. The proposed M4 and M5 fall just above doubling, and the excess shrinks as n grows, so they remain predictable as the catalogue expands.

Why the quadratic design is quadratic

In the current engine, four sub-problems of half the size are solved. The total size of the sub-problems at each level is therefore double that of the level above ($4 \times n/2 = 2n$). Data is effectively duplicated at every level, so after $\log_2 n$ levels the work is multiplied by $2^{\log n} = n$. In the redesign the total size at every level is exactly n, so nothing is repeated. This is the practical meaning of removing the replication.

(e) Analyse Best, Average and Worst Cases

The best case is the minimum number of comparisons, the worst case is the maximum, and the average case is the expected number over the assumed distribution of inputs.

► M3 – Search Version S1: Sequential Search

Case	Input condition	Comparisons
Best	The target is the first product	1
Average (successful search)	The target is equally likely to be at any of the n positions	$(n + 1)/2$
Worst	The target is the last product, or is not in the catalogue	n

The average for a successful search is $(1 + 2 + \ldots + n)/n = (n + 1)/2$. The worst-case complexity is $T(n) = \Theta(n)$.

Average comparisons when the product exists with probability p

- p = probability that the product is in the catalogue.
- $1 - p$ = probability that it is not in the catalogue (the scan checks all n products).
- $(n + 1)/2$ = average comparisons when it is present.

By the definition of expected value,

$$E[C] = p\left(\frac{n+1}{2}\right) + (1-p)\,n$$

$$E[C] = \frac{p(n+1)}{2} + n - pn = n - \frac{p(n-1)}{2}$$

Check: for $p = 1$ this gives $(n + 1)/2$, and for $p = 0$ it gives n, as expected.

Evaluation for n = 1,000,000 and p = 0.5

$$E[C] = 1{,}000{,}000 - \frac{0.5\,(999{,}999)}{2} = 1{,}000{,}000 - 249{,}999.75 = 750{,}000.25$$

A typical search therefore needs about 750,000 comparisons.

► M3 – Search Version S2: Binary Search

Case	Input condition	Comparisons
Best	The target is the middle entry of the index	1
Average	The target is found after a typical sequence of halvings (successful search)	$\approx \log_2 n - 1$
Worst	The target is absent, or found at the deepest level of the search	$\lfloor \log_2 n \rfloor + 1$

The worst-case complexity is $T(n) = \Theta(\log n)$, and the average case is also $\Theta(\log n)$. For n = 1,000,000, $\log_2 n \approx 19.93$, so the worst case is $\lfloor 19.93 \rfloor + 1 = 20$ comparisons and the average is about 19.

Comparison: in the worst case S1 needs up to 1,000,000 comparisons, but S2 needs only about 20. S2 is the better choice as long as the index is kept sorted.

Additional analysis of the search cases

Average case of binary search. Take $n = 2^k - 1$ and assume every entry is equally likely to be the target. One entry is found at comparison 1, two entries at comparison 2, four at comparison 3, and so on up to 2^{k-1} entries at comparison k. The total number of comparisons over all targets is

$$\sum_{j=1}^{k} j 2^{j-1} = (k-1)2^k + 1$$

so the average is $((k-1)2^k + 1)/(2^k - 1)$, which is close to $k - 1 = \log_2(n+1) - 1$. For $n = 15$ ($k = 4$) the total is $1 + 4 + 12 + 32 = 49$, so the average is $49/15 \approx 3.27$ comparisons, against a worst case of 4.

How the expected cost of S1 changes with *p*. Using $E[C] = n - p(n-1)/2$ with n = 1,000,000:

Probability *p* that the product exists	E[C] (comparisons)	Meaning
0.00	1,000,000.00	Unsuccessful search; the full catalogue is scanned.
0.25	875,000.12	Mostly unsuccessful searches.
0.50	750,000.25	Half the searches succeed.
0.75	625,000.38	Most searches succeed.
1.00	500,000.50	Every search succeeds; average is $(n+1)/2$.

Even when every search succeeds, S1 needs about 500,000 comparisons on average, so it never approaches the efficiency of S2. Searches for products that do not exist (typing errors, discontinued items) are the most expensive case for S1.

S1 and S2 compared for several catalogue sizes (worst case):

n	S1 worst case n	S2 worst case $\lfloor \log_2 n \rfloor + 1$	S1 ÷ S2
1,000	1,000	10	100
10,000	10,000	14	714
100,000	100,000	17	5,882
1,000,000	1,000,000	20	50,000

At the current size of 100,000 products S2 already needs 17 comparisons where S1 needs up to 100,000. At one million products the gap is about 50,000 times. S1 slows down tenfold as the catalogue grows tenfold, while S2 gains only three or four extra comparisons.

Practical considerations

- S2 needs the index to stay sorted. Inserting a new product into a sorted array costs $\Theta(n)$ shifts, so a balanced search tree or a B-tree is a better index if products are added frequently.
- S1 does not need any preparation, so it can still be used for tiny, unsorted lists such as a seller's own handful of items.
- Sorting the catalogue once costs $\Theta(n \log n)$. This pays for itself after only a few searches when the catalogue is large, because each S1 search costs $\Theta(n)$.

(f) Prove an Asymptotic Bound

The nightly report generator performs

$$f(n) = 3n^2 + 10\,n\log_2 n + 50$$

We prove that $f(n) = \Theta(n^2)$ from the definition, and then show that $f(n) \notin O(n)$.

Step 1: Definition of Big-Theta

$f(n) = \Theta(g(n))$ if there are positive constants c_1, c_2 and n_0 such that $0 \le c_1 g(n) \le f(n) \le c_2 g(n)$ for all $n \ge n_0$. Here $g(n) = n^2$.

Step 2: Lower bound

For $n \ge 1$, the terms $10n \log_2 n$ and 50 are non-negative. Dropping them gives

$$f(n) \ge 3n^2 \quad \Rightarrow \quad c_1 = 3$$

Step 3: Upper bound

For all $n \ge 1$ we have $\log_2 n \le n$, and also $1 \le n^2$. Therefore

$$10\,n\log_2 n \le 10n^2, \qquad 50 \le 50n^2$$

$$f(n) \le 3n^2 + 10n^2 + 50n^2 = 63\,n^2 \quad \Rightarrow \quad c_2 = 63$$

Step 4: Choose n_0 and conclude

Both inequalities hold for every $n \ge 1$, so $n_0 = 1$. The constants are $c_1 = 3$, $c_2 = 63$, $n_0 = 1$, and

$$3n^2 \le f(n) \le 63\,n^2 \qquad \text{for all } n \ge 1$$

By the definition, $f(n) = \Theta(n^2)$. **Numerical check** at $n = 10$: $f(10) = 300 + 10(10)(3.32) + 50 \approx 682$, and $3(100) = 300 \le 682 \le 6{,}300 = 63(100)$.

Step 5: Show that f(n) is not O(n)

Suppose $f(n) = O(n)$. Then there are constants $c > 0$ and n_1 such that $f(n) \le cn$ for all $n \ge n_1$. Since $f(n) \ge 3n^2$, this would require

$$3n^2 \le c\,n \Rightarrow 3n \le c \Rightarrow n \le \frac{c}{3}$$

But this fails for every $n > c/3$, for example $n = \max(n_1, \lceil c/3 \rceil + 1)$. A fixed constant c cannot bound a quantity that grows without limit, which is a contradiction. Hence

$$f(n) \notin O(n)$$

Conclusion: the report generator has quadratic growth, and its dominant term is $3n^2$. Its running time would grow about four times when the data doubles, not two times.

Numerical illustration of the bounds

The table checks the two inequalities $3n^2 \le f(n) \le 63n^2$ for several values of n, and also shows that $f(n)/n^2$ settles toward 3.

n	$3n^2$	$f(n)$	$63n^2$	$f(n)/n^2$	$f(n)/n$
1	3	53.0	63	53.0000	53.00
2	12	82.0	252	20.5000	41.00
10	300	682.2	6,300	6.8219	68.22
100	30,000	36,693.9	630,000	3.6694	366.94
1,000	3,000,000	3,099,707.8	63,000,000	3.0997	3,099.71
1,000,000	3,000,000,000,000	3×10^{12}	6.3×10^{13}	3.0002	3,000,199

Every row satisfies the sandwich inequality. The ratio $f(n)/n^2$ starts at 53 for $n = 1$ (the constant 50 dominates when the input is tiny) but falls toward 3, which is the leading coefficient. The ratio $f(n)/n$ grows without bound, which supports the result that f is not $O(n)$.

Limit-based confirmation

The limit test gives the same conclusion. If $\lim f(n)/g(n)$ is a finite positive constant, then $f = \Theta(g)$.

$$\lim_{n\to\infty} \frac{3n^2 + 10n\log_2 n + 50}{n^2} = \lim_{n\to\infty} \left(3 + \frac{10\log_2 n}{n} + \frac{50}{n^2}\right) = 3$$

The limit is 3, a positive finite constant, because $\log_2 n/n \to 0$ and $50/n^2 \to 0$. For the linear comparison,

$$\lim_{n\to\infty} \frac{f(n)}{n} = \lim_{n\to\infty} \left(3n + 10\log_2 n + \frac{50}{n}\right) = \infty$$

so f grows strictly faster than n, and $f(n) \notin O(n)$.

Other bounds that follow

Statement	True?	Reason
$f(n) = O(n^2)$	Yes	$f(n) \le 63n^2$ from Step 3.
$f(n) = \Omega(n^2)$	Yes	$f(n) \ge 3n^2$ from Step 2.
$f(n) = O(n^3)$	Yes	$n^2 \le n^3$, so $63n^2 \le 63n^3$; true but not tight.
$f(n) = O(n \log n)$	No	$f(n)/(n \log n) \ge 3n/\log_2 n \to \infty$.
$f(n) = \Theta(n)$	No	It is not $O(n)$, so it cannot be $\Theta(n)$.

Meaning for the platform. The $10n \log_2 n$ and 50 terms are real work, but they become negligible next to $3n^2$. At n = 1,000,000 the $3n^2$ term alone is 3×10^{12} operations, which at 10^8 operations per second is 30,000 seconds (8.3 hours). Hence the report generator would also miss a 10-minute window. Its quadratic term would need to be removed before the catalogue reaches one million.

(g) Scalability Decision

For n = 1,000,000 the running time is estimated as t = (number of basic operations) / 10^8. The nightly window of 10 minutes (600 s) allows at most 6×10^{10} operations per server. These are idealised estimates that ignore constants, memory access and network delay.

Step 1: M1 – Duplicate-Listing Detector

$$C(n) = \frac{n(n-1)}{2} = \frac{1{,}000{,}000 \times 999{,}999}{2} = 499{,}999{,}500{,}000$$

$$t = \frac{499{,}999{,}500{,}000}{10^8} = 4{,}999.995 \text{ s} \approx 83.33 \text{ minutes}$$

Decision: it fails the 10-minute limit (about 8.3 times over). **M1 must be redesigned.** The fix is to build a hash table keyed on the normalised pair (title, seller ID), so a listing is a duplicate when its key is already present. Each product is touched once, so the expected time becomes $\Theta(n)$ at the cost of $\Theta(n)$ extra memory. Sorting by the key and scanning neighbours ($\Theta(n \log n)$) is an alternative.

Step 2: M2 – API Rate Limiter

Here $\lceil \log_2 1{,}000{,}000 \rceil = 20$, because $2^{19} = 524{,}288 < 10^6 \leq 1{,}048{,}576 = 2^{20}$. So

$$A(n) = 2^{20} - 1 = 1{,}048{,}575, \qquad t = \frac{1{,}048{,}575}{10^8} \approx 0.0105 \text{ s}$$

Decision: it easily meets the limit. No redesign is needed.

Step 3: M3 – Product Search S2

A worst-case search takes $\lfloor \log_2 10^6 \rfloor + 1 = 20$ comparisons, so $t \approx 20 / 10^8 = 2 \times 10^{-7}$ seconds per search. Real latency will be higher because of memory and network overhead. **Decision:** it meets the requirement by a wide margin, so keep the sorted index and the binary search. (For comparison, a worst-case S1 scan of 10^6 comparisons takes 0.01 s, which is acceptable for one query but costly across millions of queries.)

Step 4: M4 – Current Recommendation Engine

Solving $T(n) = 4T(n/2) + n$ with $T(1) = 1$ for powers of two gives $T(n) = 2n^2 - n$. The order-of-growth estimate uses n^2:

$$\text{Operations} \approx (10^6)^2 = 10^{12}, \qquad t \approx \frac{10^{12}}{10^8} = 10{,}000 \text{ s} \approx 166.7 \text{ minutes}$$

Including the constant 2 from the exact solution the workload is about 2×10^{12} operations, or about 20,000 s (333 minutes). Either way the engine is 17 to 33 times over the window. **Decision:** it fails and must be redesigned by removing the replicated sub-problems.

Step 5: M4 – Proposed Recommendation Engine

For $T(n) = 2T(n/2) + n$, the leading term is $n \log_2 n$ (the exact solution is $n \log_2 n + n$ when $T(1) = 1$):

$$n\log_2 n = 10^6 \times 19.93157 \approx 19{,}931{,}569, \qquad t \approx \frac{1.993 \times 10^7}{10^8} \approx 0.199 \text{ s}$$

Decision: it meets the limit comfortably. About 21 million operations including the linear leaf term gives roughly 0.21 s, so the conclusion does not change.

Step 6: M5 – Sales Analytics Engine

The tree sum in part (d) gives $T(n) = n + nL(L + 1)/2$ with $L = \log_2 n \approx 19.93$, which is about 2.09×10^8 operations. Using the Θ-estimate $n(\log_2 n)^2$ as a cautious upper figure:

$$n(\log_2 n)^2 = 10^6 \times (19.93157)^2 \approx 3.97 \times 10^8, \qquad t \approx \frac{3.97 \times 10^8}{10^8} \approx 3.97 \text{ s}$$

Decision: M5 meets the limit (about 2 to 4 seconds). It should still be monitored, and repeated sorting of already sorted data should be avoided.

Step 7: Comparison of all modules (n = 1,000,000)

Module / design	Estimated operations	Estimated time	10-minute limit
M1 – Duplicate detector	4.999995×10^{11}	83.33 min	Fails
M2 – Rate limiter	1,048,575	0.0105 s	Meets
M3 – S2 binary search (per search)	about 20 comparisons	2×10^{-7} s	Meets
M4 – Current design	10^{12} to 2×10^{12}	166.7 to 333 min	Fails
M4 – Proposed design	about 1.99×10^7	0.199 s	Meets
M5 – Sales analytics	2.09×10^8 to 3.97×10^8	2.1 to 3.97 s	Meets

The S2 figure is per search, while the others are for the whole nightly routine. The M1 and M4 (current) times are order-of-growth estimates, not exact predictions.

Step 8: Growth from today's catalogue to the expected catalogue

Comparing the current size (n = 100,000) with the expected size (n = 1,000,000) shows how much each module slows down as the catalogue grows tenfold.

Module / design	Operations at $n = 10^5$	Time at $n = 10^5$	Operations at $n = 10^6$	Slow-down factor
M1 – Duplicate detector	4,999,950,000	50.0 s	4.999995×10^{11}	about 100×
M2 – Rate limiter	$2^{17} - 1 = 131{,}071$	0.0013 s	1,048,575	8×
M3 – S2 (per search)	17 comparisons	1.7×10^{-7} s	20 comparisons	about 1.2×
M4 – Current design	about 10^{10} to 2×10^{10}	100 to 200 s	10^{12} to 2×10^{12}	about 100×
M4 – Proposed design	$\approx 1.66 \times 10^6$	0.017 s	$\approx 1.99 \times 10^7$	about 12×
M5 – Sales analytics	$\approx 1.5 \times 10^7$ to 2.8×10^7	0.15 to 0.28 s	$\approx 2.1 \times 10^8$ to 4.0×10^8	about 14×

Interpretation: a tenfold increase in data makes the quadratic modules (M1 and the current M4) about 100 times slower, while the linear and n log n-type modules slow down by only about 8 to 14 times and the logarithmic search barely changes. Both quadratic modules already take 50 to 200 seconds today, so they are the first to hit the 10-minute window as the catalogue grows. Using n^2 as the guide, they would reach 600 s at roughly $n \approx 346{,}000$ (M1, since $n^2/2 = 6 \times 10^{10}$) and $n \approx 173{,}000$ to 245,000 (current M4, from $2n^2 = 6 \times 10^{10}$ and $n^2 = 6 \times 10^{10}$), well before one million products.

Runtime comparison chart

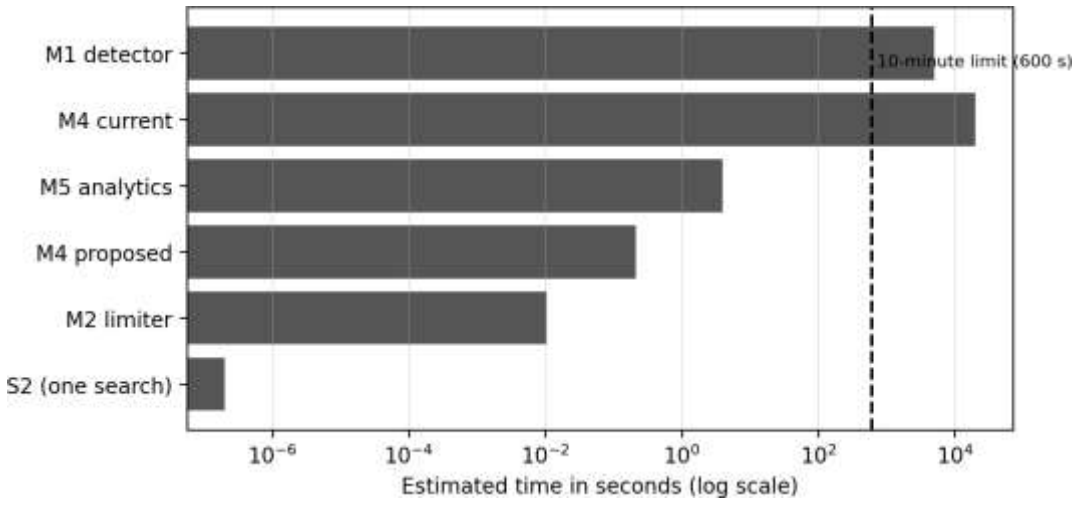


Figure 2: Estimated running times at n = 1,000,000. Bars to the right of the dashed line fail the nightly window.

Impact at the current catalogue size

The same formulas can be applied to today's catalogue of n = 100,000 products to show how close each module already is to its limit.

Module / design	Operations at n = 100,000	Time	Operations at n = 1,000,000	Time	Slowdown
M1 detector	5×10^9	50 s	5×10^{11}	5000 s	100×
M2 limiter	131,071	0.001311 s	1,048,575	0.01049 s	8×
M4 current	2×10^{10}	200 s	2×10^{12}	2e+04 s	100×
M4 proposed	1,760,964	0.01761 s	2.09×10^7	0.2093 s	12×
M5 analytics	1.47×10^7	0.1472 s	2.1×10^8	2.096 s	14×

Today M1 takes about 50 seconds and the current M4 about 200 seconds. Both are inside the 600-second window now, which is why no problem has been noticed. A tenfold increase in data multiplies their run time by about 100, which pushes them far beyond the window. The linear and near-linear modules slow down by only about 10 to 13 times.

Largest catalogue each module can handle in 10 minutes

Solving (operations) $\leq 6 \times 10^{10}$ for n shows how much headroom each design has before it breaks the nightly limit.

Module / design	Cost formula used	Largest n within 600 s
M1 detector	$n(n-1)/2$	346,410
M4 current	$2n^2 - n$	173,205
M5 analytics	$n + n\mathrm{L}(\mathrm{L}+1)/2, \mathrm{L} = \log_2 n$	about 155,000,000
M4 proposed	$n \log_2 n + n$	about 1,880,000,000
M2 limiter	$2^{\lceil \log_2 n \rceil} - 1$	about 3.4×10^{10} (2^{35})

The current M1 and M4 designs can survive only about 350,000 and 170,000 products respectively, so they would already fail before the catalogue reaches the expected size. The redesigned M4 and the existing M5 have room for billions of records, so they will not be bottlenecks for the foreseeable future.

Pseudocode for the recommended redesign of M1

```
seen <- empty hash table
FOR i <- 0 TO n-1 DO
    key <- Normalize(Product[i].title) + '|' + Product[i].seller
    IF key IN seen THEN
        MarkDuplicate(i)
    ELSE
        INSERT key INTO seen
    END IF
END FOR
```

Each product is hashed and looked up once, so the expected time is $\Theta(n)$ and the extra space is $\Theta(n)$. For one million products this is about a million operations (times a small hashing constant), far below the 6×10^{10} budget. The trade-off is memory: a million keys of 50 to 100 bytes each need roughly 50 to 100 MB, which is acceptable on a normal server. The Normalize step (lower-casing, trimming spaces) also helps catch near-duplicates that the original exact comparison would miss.

Space complexity of all modules

Module / design	Time	Auxiliary space	Remark
M1 current	$\Theta(n^2)$	$\Theta(1)$	Only loop counters.
M1 hash-based	$\Theta(n)$ expected	$\Theta(n)$	Stores one key per product.
M2	$\Theta(n)$	$\Theta(1)$	Only i and j.
M3 S1	$O(n)$	$\Theta(1)$	Single scan.
M3 S2 (iterative / recursive)	$\Theta(\log n)$	$\Theta(1)$ / $\Theta(\log n)$	Needs a sorted index.
M4 current / proposed	$\Theta(n^2)$ / $\Theta(n \log n)$	$\Theta(n)$ typical for merging	Merge buffer plus recursion depth $\Theta(\log n)$. The replicated partitions need more memory in the current design.
M5	$\Theta(n \log^2 n)$	$\Theta(n)$ typical for merge-based sorting	Temporary arrays for sorting.

The M4 and M5 memory figures are the usual values for merge-based divide-and-conquer methods, since the scenario gives only their time recurrences. Exact memory use depends on the implementation.

Suggested implementation roadmap

Priority	Action	Module	Expected benefit
1 (urgent)	Replace nested comparison with hash-based grouping	M1	83 minutes down to seconds; removes the quadratic risk.
1 (urgent)	Remove replicated partitions; adopt $2T(n/2) + n$ design	M4	From hours to under one second at one million records.
2	Re-examine the nightly report generator's $3n^2$ term	Report generator	Avoids a further 8-hour job at one million products.
3	Keep the sorted index and consider a balanced tree for insertions	M3 S2	Keeps search at about 20 comparisons.
4	Profile M5 monthly and avoid repeated sorting	M5	Early warning if data grows beyond expectation.
4	No change	M2	Already linear.

Limitations of this audit

- Counts ignore constant factors, which can differ by a large amount between implementations. A well-tuned $\Theta(n^2)$ routine can be faster than a poorly written $\Theta(n \log n)$ one for small n, but not at one million.
- The 10^8 operations per second figure is an assumption given by the question. Real throughput depends on caching, disk and network.
- Recurrences were solved for powers of two. For n = 1,000,000 (not a power of two) the results are close approximations, as the asymptotic classes remain valid.
- Expected hash-table performance assumes a good hash function and a bounded load factor.

◆ Final Scalability Recommendations

1. **Redesign M1.** Replace the nested loops with hash-based grouping on (title, seller ID). Time falls from $\Theta(n^2)$ to expected $\Theta(n)$, bringing the run from about 83 minutes to a few seconds, at the cost of extra memory.
2. **Redesign M4.** Remove the replicated partitions so the recurrence becomes $2T(n/2) + n$. Complexity falls from $\Theta(n^2)$ to $\Theta(n \log n)$, from over 2.7 hours to under a second.
3. **Retain M2.** Its $\Theta(n)$ cost is about 0.01 s at one million requests.
4. **Retain M3 with S2.** Binary search takes about 20 comparisons, provided the product index is kept sorted as products are added.

5. **Retain and monitor M5.** Its $\Theta(n \log^2 n)$ cost fits the window, but profiling should continue as data grows. The report generator's $\Theta(n^2)$ cost from part (f) should also be watched, since it will grow 100 times when n grows 10 times.

◆ Overall Conclusion

The audit shows that the choice of algorithm decides whether ShopSphere can handle one million products. Only the duplicate-listing detector (M1) and the current recommendation engine (M4) exceed the 10-minute window, because both are quadratic. Hash-based duplicate detection and the non-replicated recommendation design bring both within the limit with a large margin. M2, M3 (S2) and M5 can be kept, with M5 monitored. With these changes the platform can grow its catalogue while keeping search, recommendations, duplicate detection and analytics efficient.

References

- Cormen, T. H., Leiserson, C. E., Rivest, R. L. and Stein, C., *Introduction to Algorithms*, MIT Press (chapters on growth of functions, recurrences and the Master Theorem).
- Levitin, A., *Introduction to the Design and Analysis of Algorithms*, Pearson (chapters on analysis framework, brute force and divide-and-conquer).